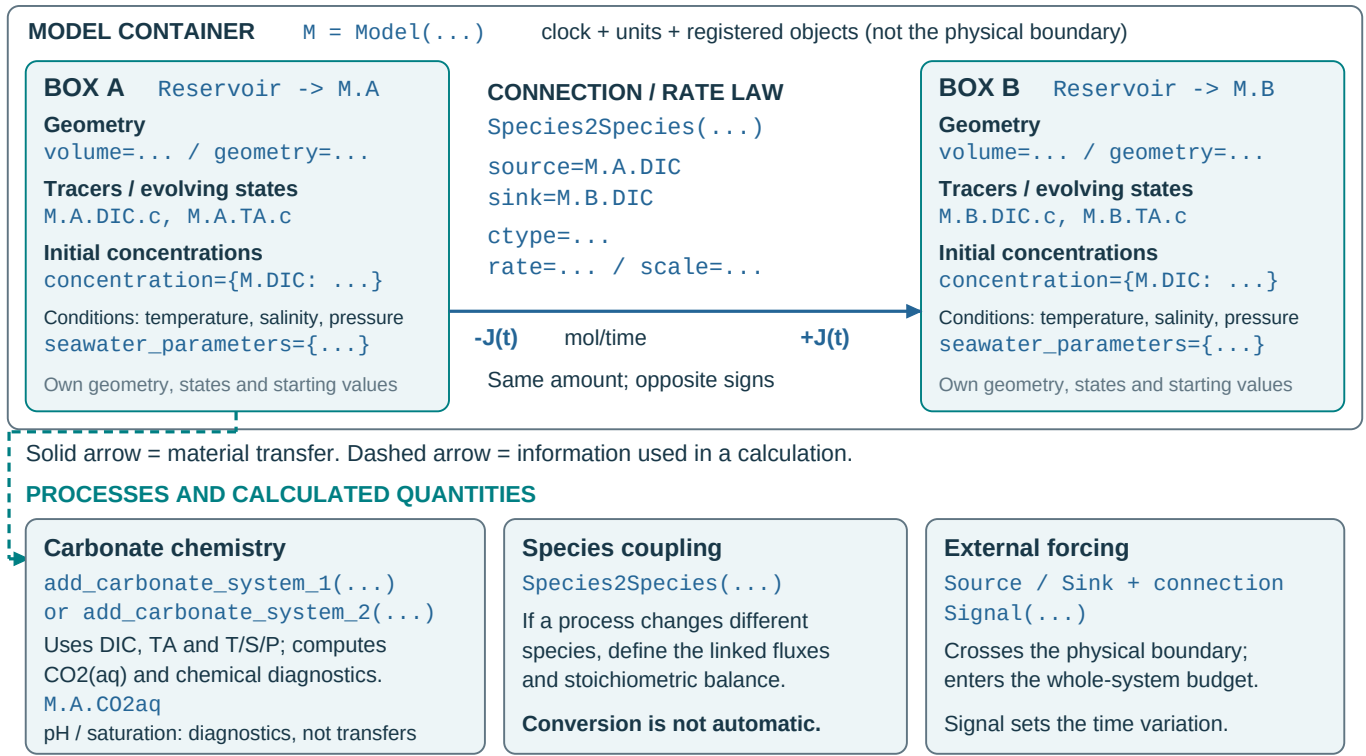


From conceptual model to code

ESBMTK practicals 01-04 | Page 1: translate the science

Aim: explain how boxes, arrows and assumptions become a working model. This sheet is optional lookup support; syntax memorisation is not required. Use **predict -> map -> run -> check -> explain**.

What does each part of the diagram become?



M.DIC = species definition; M.A.DIC = that species in A; .c = concentration time series.

Read boxes, arrows, then the legend. This generic map shows one internal transfer. A water-transport model also needs water balance and all transported tracers. Notebook 01 applies this visual language to its worked example.

One arrow, from equation to native ESBMTK code

Unrelated example: sealed, well-mixed tanks with passive O₂: no reactions or air exchange. A device moves **0.1 mol/yr** from A to B for five years, without moving water. This artificial transfer is not a gas-exchange or water-flow law.

System boundary: [A: 1000 L | O₂] -- J = 0.1 mol/yr --> [B: 2000 L | O₂]

For concentration c in mol/L and fixed volume V in L, inventory N = V c (mol).

$dN_A/dt = -J$; $dN_B/dt = +J$; $d(N_A + N_B)/dt = 0$. The same arrow enters both balances with opposite signs. Concentration changes differ because the volumes differ.

```
from esbmtk import Model, Reservoir, Species2Species
from model import run_model # supplied course runner

M = Model(stop="5 yr", max_timestep="0.1 yr", element=["Oxygen"],
          mass_unit="mol", concentration_unit="mol/l")
Reservoir(name="A", register=M, volume="1000 l",
          concentration={M.O2: "2 mmol/l"})
Reservoir(name="B", register=M, volume="2000 l",
          concentration={M.O2: "0.5 mmol/l"})
transfer = Species2Species(source=M.A.O2, sink=M.B.O2,
                          ctype="regular", rate="0.1 mol/yr",
                          id="tracer_transfer")

run_model(M)
total = 1000 * M.A.O2.c + 2000 * M.B.O2.c # mol, at saved times
```

M.O2 identifies the species; M.A.O2 its state in A. .c is in mol/L: [0] first, [-1] last. A/B initially contain 2/1 mol; after five years, 1.5/1.5 mol. total remains 3 mol. Stop before A empties.

Choices: volumes, initial concentrations, endpoints and rate. register=M attaches the box; name and id identify objects. Optional reference: run from the repository root in ESBMTK314; no extra assignment.

Reading and running the practicals

ESBMTK practicals 01-04 | Page 2: navigate the implementation

Know where to spend your attention

Choose and explain: complete marked scientific choices and derivations; justify the mapping, predictions and interpretations. Constructor templates are supplied.

Understand and run: trace worked construction and read tables, graph checks, budgets and plots.

Supplied implementation: run imports, loops, conversions, chemistry/sediment wiring and numerical/plotting internals as provided. These practicals are ungraded; focus on understanding the scientific choices and interpreting their results.

Follow the execution order

Load inputs -> construct reservoirs -> connect processes -> set starting state -> run -> check -> interpret. ESBMTK advances coupled balances through time. In 03/04, a stationary restart replaces the initial values used in construction; it supplies state, not connections.

After edits, rerun setup and dependent construction/run/check cells in order. Editing a dictionary does not rebuild objects; defining a function does not run it. Restarting the kernel and executing top to bottom removes stale state; complete exercise placeholders first. Changing geometry, chemistry or baseline rates in 03/04 requires a new stationary restart and matched control.

Python patterns you will meet

Pattern	Read it as
<code>source=M.A.02</code>	Named argument: the O2 state in A is the source.
<code>row['volume']; getattr(M, box_name)</code>	Read a dictionary field; retrieve an attribute by name.
<code>{M.O2: '2 mmol/l'}; [M.DIC, M.TA]</code>	Dictionary: keys mapped to values; list: grouped items.
<code>for row in rows; **settings</code>	Repeat an indented mapping; pass a dictionary as named arguments.
<code>def build_case(...); build_case(...)</code>	Define a recipe; call it to construct a case.

Where the science and supporting code live

Location	Role in your route
Student notebooks 01-04	Scientific choices, experiment and interpretation.
<code>teaching_config.py; simple_models.py</code>	Shared 01/02 settings; construction/inventory helpers.
<code>data/Boudreau_2010/model_definition.xlsx</code>	03/04 baseline inputs, displayed in notebook tables.
<code>model_inputs.py; presets.py</code>	Read/validate inputs and prepare configurations; you still construct the graph in 03.
<code>model.py; teaching_audits.py; teaching_plots.py</code>	Reusable model/runner, complete-model budgets and figures.

`initialize_reservoirs` and `create_bulk_connections` repeat native construction from records. Template keys: `c` = concentrations, `g` = geometry, `sp` = species, `ty` = type, `sc` = scale. Follow each notebook's call pattern.

Before trusting a result

Trace the arrow: check endpoints, species, direction and units. Water carries source concentrations; gas exchange can reverse sign. Internal transfers cancel in the total inventory; boundary fluxes remain.

Check quantities: flux = amount/time; inventory = amount. Tanks: $\text{mol/L} \times \text{L}$. Ocean DIC: $\text{mol/kg} \times \text{water mass}$ (ESBMTK density $\times$ volume); TA uses alkalinity equivalents. Atmospheric carbon: CO_2 mole fraction $\times$ air mole inventory (convert ppm to a fraction). Integrating an input flux gives its added amount.

Read failures: `NotImplementedError` = unfinished exercise; missing name = check setup; missing key = check dictionary fields. Failed budget: check units and connections before interpreting plots.

Explain evidence: passing budgets supports implementation verification; a fitted target is not independent validation. The final point need not be equilibrium. Trace one assumption through its code, balance and check.